Table of Content

Abstract....1

Introduction 2

1.1 Introduction 2

1.2 Objectives 3

1.3 Methodology 4

1.3.1 Data Collection 4

1.3.2 Data Preprocessing 4

1.3.2.1 Data Loading 4

1.3.2.2 Exploratory Data Analysis (EDA) 4

1.3.3 Model Building 4

1.3.3.1 Data Splitting 4

1.3.3.2 Model Development 5

1.3.4 Model Evaluation 6

Result and Discussion....6

2.1 Dataset Description 6

2.2 Data Preprocessing....8

2.2.1 Import necessary libraries 8

2.2.2 Import the file entitled “online_shoppers_intention” 8

2.2.3 Checking missing values....8

2.2.4 Making sure data is consistent

2.3 Exploratory Data Analysis (EDA) 12

2.3.1 Summary of Data 12

2.3.2 Distribution of Product Related Duration 12

2.3.3 Distribution of Administrative Duration....13

2.3.4 Distribution of Exit Rates 14

2.3.5 Distribution of Revenue 15

2.3.6 Count of Visitor Type....16

2.3.7 Count of Visits by Month....17

2.3.8 Comparison of Relationship Between Revenue by Visitor Type and by Weekend 18

2.3.9 Relationship Between Revenue and Month 19

2.3.10 Relationship Between Revenue and Special Day 21

2.3.11 Proportion of Visitor Type by Weekend 22

2.3.12 Correlation Between Variables 23

2.3.13 Relationship between Exit Rates and Bounce Rates by Revenue 24
2.3.14 Relationship between Product Related and Product Related Duration..... 25
2.3.15 Trend Line for Visitor Type Based on Months..... 26
2.4 Preparation Before Building Models..... 27
2.4.1 Checking type of variables 27
2.4.2 Checking completeness of data..... 28
2.4.3 Converting non-numerical variables to factors 28
2.4.4 Normalizing numerical columns for standardization 29
2.4.5 Splitting data to training and testing data 29
2.5 Building Supervised Learning Models 29
2.5.1 Decision Tree 30
2.5.2 Random Forest 32
2.5.3 SVM (Support Vector Machine) 34
2.5.4 Naïve-Bayes Classifier 35
2.5.5 KNN Algorithm 36
2.5.6 Logistic Regression 38
2.5.7 Comparing Supervised Learning Models..... 40
Closing 41
3.1 Summary 41
3.2 Conclusion 42

Abstract

The swift growth of e-commerce has increased the need for businesses to understand and forecast online buyers' purchase patterns in order to improve their marketing tactics and increase conversion rates. Our goal in this study is to use the Online Shoppers Purchasing Intention Dataset from the UC Irvine Machine Learning Repository to create a strong predictive model that will predict the purchases made by online shoppers. 84.5% (10,422) of the 12,330 sessions in this dataset fall into the negative category (no purchase), whereas 15.5% (1,908) fall into the positive category (completed purchase). The disparity between these classes makes developing a precise prediction model extremely difficult.

We use a hybrid strategy to address this problem, using many machines learning algorithms, including K-Nearest Neighbours (KNN), Random Forest, Logistic Regression, Decision Tree, Support Vector Machines (SVM), and Naive-Bayes Classifier. A variety of characteristics, including session time, page views, bounce rate, exit rate, and closeness to special days, are used to train the models. The dataset also goes through preprocessing, which includes dividing it into training (70%) and testing (30%) subsets, normalising numerical variables, and encoding categorical characteristics.

The importance of using hybrid machine learning models to forecast online buyers' behaviour is shown by this study, which offers insightful information for maximising marketing campaigns and boosting sales conversions. In order to increase predictive accuracy, particularly in datasets with class imbalances, the study also emphasises the significance of efficient feature selection, data preparation, and model evaluation.

Keywords: Machine Learning, Predictive Modelling, E-commerce, Online Shoppers, Purchasing Intention, Logistic Regression, Support Vector Machines, Decision Trees, Random Forest, KNN, Naive Bayes, Class Imbalance.

Chapter 1. Introduction

1.1. Introduction

The rapid growth of e-commerce not only revolutionized the manner in which business and customers communicate but also modified the manner in which marketing, customer interaction, and sales strategies are employed. In this day and age, where retail is vastly through e-commerce channels, it becomes very relevant that businesses understand who their customers are in order to realize the best marketing optimization, conversion rates, and customer satisfaction. One of the major problems in e-commerce is the possibility that a shopper will make a purchase after searching. It's important for businesses to be able to predict the buying behaviour of their potential customers, as this enables them to design their market offerings in a way that enhances efficiency in resource allocation, customer targeting, and sales maximization. The current research will therefore seek to build a predictive model on analysing the purchase intentions of online shoppers using data from the Online Shoppers Purchasing Intention dataset available in the UC Irvine Machine Learning Repository. The dataset includes 12,330 sessions of each consumer's interaction on an e-commerce site for 1 year. Of this, a large proportion, 84.5% (10,422) are negative class samples indicating that the purchase was not completed, while the remaining 15.5% (1,908) are positive class samples where the session ended with the purchase of the intended items. This is a class imbalance problem and has become one of the major obstacles to building an accurate predictive model for this problem, since most machine learning approaches tend to be biased toward the larger class.

This dataset contains 17 features, including 10 numerical and 7 categorical variables, which are necessary for e-commerce shopping behaviour analysis. The features include metrics of the number of visits on various page types "Administrative", "Product Related", and "Informational", duration for which each page is opened, bounce rate, exit rate, page value, and special day pointers. Target Variable: The target variable of this model is the "Revenue" column or, rather, the class label for whether a purchase was made or not. In practice, there are other variables useful in carrying out analysis on the pattern between some variables such as time spent on the site and type of information, which comprises the number of the page visited, first time logged in, and how many times each has logged in before. Besides, the features "Month" and "Visitor Type" can give information about the time that the session occurred and the site's familiarity by the user.

A variety of machine learning algorithms is needed in order to be able to predict purchasing behaviour from such a heterogeneous dataset including numerical and categorical variables while also addressing the class imbalance. These models involve several machine learning algorithms, which are applied in tandem, including but not limited to Random Forest, Logistic Regression, decision trees, SVM, Naive-Bayes Classifier, and K-Nearest Neighbors. Each model possesses

specific advantages in dealing with non-linear relationships inherent in the data and boosts the accuracy of the predicted estimates. For example, Random Forest offers an advantage in handling a high-dimensional amount of strong interacting features; whereas, SVM benefits due to its ability of finding optimal hyperplanes in order to classify high-dimensional features. Further, we are going to include the multistage preprocessing solution which consists of features normalization, categorical features preprocessing, and splitting data on training and testing which, finally allows training of proper model training and validation of their results. The data was divided into training data consisting of 70% of the data and test data consisting of 30%, where the models were developed based on the training data and were assessed using the test data. Therefore, the performance of each model can be assessed independently and their forecasting power can be evaluated easily.

The main intention of this research work is to investigate the performance of hybrid machine learning models in modelling and predicting the purchase intention of customers that shop online, and also finding characteristics that influence decision making on purchase. Also, we present strategies for class imbalance problems including model selection and balanced metric performance measures. The aim of this particular study is to help further the growing literature of e-commerce analytics by providing a way different models of machine learning can be leveraged to sift through massive sets of complex data to offer insights that can facilitate an organization in enhancing the rate of conversion of their marketing strategies and sales conversion rates.

We are also interested in the role of feature selection and data preprocessing in the development of a suitable predictive model. The capability to foresee whether or not a customer buying goods on online services would make a purchase has far-reaching effects in e-commerce business, such as making efficient advertisement methods, better segmentation of sales promotions, and even effective use of internet traffic. In the future, such predictive systems could provide the foundation of automation decision-making mechanisms impacting how customers interact with the businesses in the online environment.

Keywords: Machine Learning, Predictive Modelling, E-commerce, Online Shoppers, Purchasing Intention, Logistic Regression, Support Vector Machines, Decision Trees, Random Forest, KNN, Naive Bayes, Class Imbalance.

1.2. Objectives

1.2.1. The aim is to do data preprocessing and EDA with R to find patterns in trends in the dataset.

1.2.2. To compare the performance of different supervised learning models on online shopping behaviour comprehensively.

1.2.3. To implement a prediction of online shoppers purchasing intention using various machine learning algorithms available in R.

1.2.4. To identify the most important features influencing purchasing decisions using Random Forest and Decision Tree models.

1.2.5. To provide a comprehensive comparison of the performance of supervised learning models in the context of online shopping behaviour.

1.2.6. To analyse feature importance using R's built-in functions and visualization tools

1.2.7. To implement ensemble learning techniques in R to improve the prediction accuracy.

1.3. Methodology

1.3.1 Data Collection

This journal's dataset, "Online Shoppers Purchasing Intention," was sourced from UC Irvine machine learning database. The dataset consists of many variables relative to online shopping which encompasses how users interacted and their demographic and a binary variable that indicates the purchase behaviour of a shopper.

1.3.2 Data Preprocessing

1.3.2.1 Data Loading

Reading raw data from sources using the `read_csv()` function. Verifying file paths and data formats, and handling if there any potential file reading errors.

1.3.2.2 Exploratory Data Analysis (EDA)

The data is being load to the R script. Then, `str()` function has been used to understand properties of the dataset. These provide summaries of categorical variables such as null value occurrences. The `sum(is.na())` function checks whether a null value exists. The variables such as Month, VisitorType, Revenue were transformed into factor data types for ease of analysis.

1.3.3 Model Building

The model was reshaped and all the numeric variables were scaled to avoid them having different weight effects on model training using the `scale()` function.

1.3.3.1 Data Splitting

The data was then divided into training and test sets using the 70-30 split. The aim was to sample in accordance with the target variable. Hence, the `createDataPartition()` function from the caret package was used.

1.3.3.2 Model Development

Various machine learning models were developed and tested for their performance:

Random Forest:

A Random Forest model was built using the randomForest package, specifying 100 trees. Predictions were made on the test dataset, and a confusion matrix was created to assess the model's accuracy.

Logistic Regression:

A logistic regression model was developed using the `glm()` function with a binomial family. Predictions were generated and a confusion matrix was created to evaluate the model's performance.

Decision Tree:

A decision tree was developed using the rpart package. The tree is visualized using the `rpart.plot()` function, and a prediction is made on the test set.

Support Vector Machine (SVM):

An SVM model using the e1071 package with a linear kernel has been developed. Predictions are made and a confusion matrix is drawn to judge the accuracy of the developed model.

Naive Bayes:

A Naive Bayes classifier was developed using the `naiveBayes()` function from the e1071 package. Predictions were made on the test dataset, and a confusion matrix was generated to evaluate the model's performance.

K-Nearest Neighbors (KNN):

The KNN algorithm was implemented using the class package. The data was normalised to ensure that all the attributes had equal impact in the computation of distance. The best value of 'k' was identified as that number which had the highest accuracy for values of k from 1 to 20.

1.3.4 Model Evaluation

For each model, the performance was ascertained using confusion matrices which helped report on the accuracy, precision, recall, and the F1-score in relation to each classification model. Once the overall accuracy for each model had been established, a comparison was made in order to be able to identify the best technique for estimating the purchasing intentions of online shoppers.

Chapter 2. Result and Discussion

2.1. Dataset Description

Dataset Title = Online Shoppers Purchasing Intention Dataset

CSV Title = online_shoppers_intention

Date Uploaded = 8/30/2018

Dataset Characteristics = Multivariate

Subject Area = Business

Number of Rows = 12331

About:

The dataset consists of feature vectors belonging to 12,330 sessions. It was formed so that each session would belong to a different user in a 1-year period to avoid any tendency to a specific campaign, special day, user profile, or period. Of the 12,330 sessions in the dataset, 84.5% (10,422) were negative class samples that did not end with shopping, and the rest (1908) were positive class samples ending with shopping.

Variables in this dataset are as follows:

No.	Variable Name	Type	Description
1.	Administrative	Integer	Number of administrative pages visited by the user during the session.
2.	Administrative_Duration	Integer	Total time spent (in seconds) on administrative pages during the session.
3.	Informational	Integer	Number of informational pages visited during the session.
4.	Informational_Duration	Integer	Total time spent (in seconds) on informational pages during the session.

5.	ProductRelated	Integer	Total time spent (in seconds) on product-related pages during the session.
6.	ProductRelated_Duration	Continuous	Percentage of visitors who left the website after viewing only a single page.
7.	BounceRates	Continuous	Percentage of pageviews after which the user exited the website.
8.	ExitRates	Continuous	Average value of a page visited by the user during the session, calculated based on the e-commerce revenue.
9.	PageValues	Integer	Average value of a page visited by the user during the session, calculated based on the e-commerce revenue.
10.	SpecialDay	Integer	Indicates closeness of the session to a special day, such as Mother's Day, Valentine's Day, etc. The range is 0 (not close) to 1 (very close).
11.	Month	Categorical	Month in which the session occurred.
12.	OperatingSystems	Integer	Numeric representation of the operating system used during the session.
13.	Browser	Integer	Numeric representation of the browser used during the session.
14.	Region	Integer	Numeric representation of the geographical region of the user.
15.	TrafficType	Integer	Numeric representation of the type of traffic source.
16.	VisitorType	Categorical	Categorical variable indicating if the visitor is a "Returning_Visitor" or "New_Visitor."
17.	Weekend	Binary	Binary variable indicating whether the session occurred on a weekend. The values are TRUE (weekend) or FALSE (weekday).
18.	Revenue	Binary	Binary target variable indicating whether the session ended with a

			purchase. The values are TRUE (purchase made) or FALSE (no purchase).

2.2. Data Preprocessing

2.2.1. Import necessary libraries

```
library(tidyverse)
```

Uses = data manipulation, exploration, and visualization.

```
library(janitor)
```

Uses = Cleans column names to make them consistent and R-friendly

```
library(dplyr)
```

Uses = Functions like mutate() (add/modify columns), filter() (filter rows), select() (choose columns), and arrange() (sort rows).

```
library(ggplot2)
```

Uses = build plots layer by layer.

```
library(car)
```

Uses = companion to applied regression, used for statistical analysis and diagnostics.

2.2.2. Import the file entitled “online_shoppers_intention”

```
data <- read.csv("online_shoppers_intention.csv")
```

2.2.3. Checking missing values

```
sum(is.na(data)) #data has no NA
```

More information = Checking for missing values is essential to maintain data integrity, as missing data can distort analyses and lead to biased results. It allows for proper handling, such as imputation or exclusion, ensuring that statistical models and machine learning algorithms are trained on complete and accurate data. In this case, there is no missing values and therefore does not need any treatment.

2.2.4. Making sure data is consistent

```
data <- data %>% clean_names()
```

More information = `clean_names()` can ensure the column to have consistent data format because it converts names to lowercase, and replaces space or special characters with underscores.

2.2.5. Changing variable to necessary data type

```
data <- data %>%
mutate(across(where(is.character), as.factor))
```

More information = Converting character variables like “Month” and “VisitorType” to factors in R is crucial because factors explicitly define categorical data, making them more memory-efficient and suitable for statistical modelling. Factors ensure proper handling in regression and visualization tools like ggplot2, where categorical variables are treated appropriately, and levels can be ordered logically. Additionally, factors reduce errors by restricting entries to predefined levels and improve data visualization by maintaining meaningful groupings. This conversion aligns data representation with its intended categorical use, enabling accurate analysis and interpretation.

2.2.6. No need to check for duplicates

Checking and removing duplicates in this dataset is unnecessary because each row represents a unique session over a one-year period, ensuring no overlap between users. The absence of identifying variables like “session IDs” or “user IDs” means that similar rows do not indicate redundancy but rather valid, separate sessions with similar attributes. E-commerce behaviour often follows repetitive patterns, such as similar bounce rates or page durations, especially during seasonal events, and these repetitions reflect real-world occurrences. Removing apparent duplicates could result in the loss of legitimate data, introducing bias, especially given the dataset's imbalance, where purchases are underrepresented. Retaining all sessions preserves the diversity and integrity of the data, ensuring accurate insights and predictions.

2.2.7. Checking outliers

Outliers may be genuine but rare events or errors in data collection. Depending on the context, they can be addressed by removing, transforming, or correcting the values. Treating outliers is crucial for ensuring the integrity of analyses, as they can disproportionately influence metrics like mean, variance, and regression coefficients. Identifying and addressing outliers helps in building more reliable models, improving predictions, and avoiding misleading conclusions.

```
# Focus on numerical columns only
numeric_columns <- data %>%
  select(where(is.numeric))

# Detect Outliers Using IQR Method
detect_outliers <- function(x) {
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_value <- Q3 - Q1
  lower_bound <- Q1 - 1.5 * IQR_value
  upper_bound <- Q3 + 1.5 * IQR_value
  outliers <- x[x < lower_bound | x > upper_bound]
  list(
    lower_bound = lower_bound,
    upper_bound = upper_bound,
    outliers = outliers,
    count = length(outliers)
  )
}

# Checking Outliers for every numerical column
outlier_results <- lapply(numeric_columns, detect_outliers)

# Outliers and counts in every column
for (col_name in names(outlier_results)) {
  cat("\nOutliers for", col_name, ": ")
  cat("outliers:", outlier_results[[col_name]]$count)
}
```

The outputs are shown below:

- Outliers for Administrative: outliers: 404
- Outliers for Administrative_Duration: outliers: 1172
- Outliers for Informational: outliers: 2631
- Outliers for Informational_Duration: outliers: 2405
- Outliers for ProductRelated: outliers: 987
- Outliers for ProductRelated_Duration: outliers: 961

```
-   Outliers for BounceRates: outliers: 1551
-   Outliers for ExitRates: outliers: 1099
-   Outliers for PageValues: outliers: 2730
-   Outliers for SpecialDay: outliers: 1251
-   Outliers for OperatingSystems: outliers: 111
-   Outliers for Browser: outliers: 4369
-   Outliers for Region: outliers: 511
-   Outliers for TrafficType: outliers: 2101

# See Outliers from Boxplot
numeric_columns %>%
  pivot_longer(cols = everything(), names_to = "Variable",
  values_to = "Value") %>%
  ggplot(aes(x = Variable, y = Value)) +
  geom_boxplot(fill = "lightblue", color = "black",
  outlier.color = "red") + theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  ggtitle("Boxplots of Outliers")
```

The output is shown below:

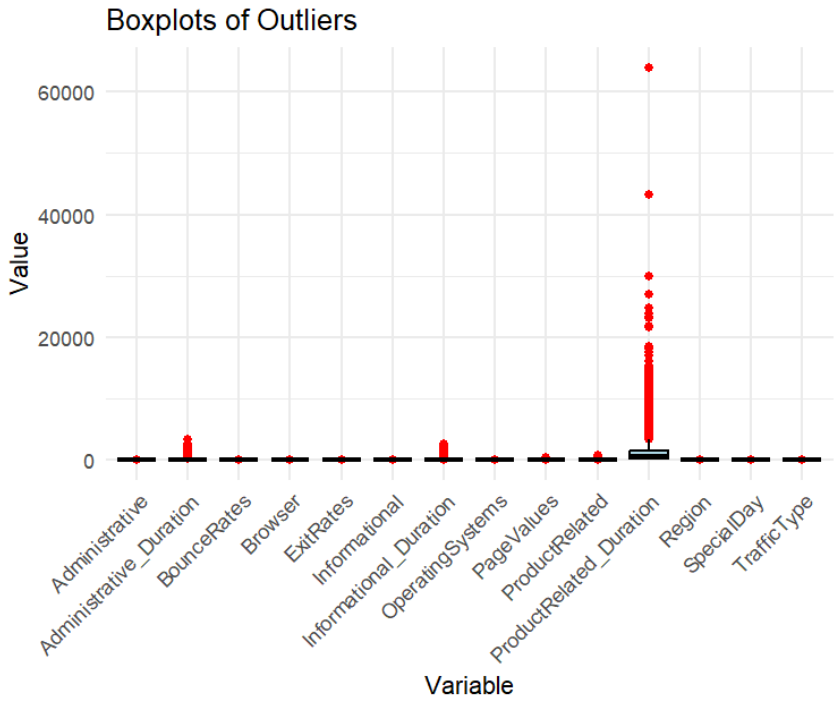


Image 3.2.1

Boxplots of Outliers

Interpretation:

Each box represents the interquartile range (IQR), where the central line indicates the median, and the whiskers extend to the minimum and maximum values within 1.5 times the IQR. Points outside of the whiskers are considered outliers. The outliers of the data are marked as red dots and we can see a lot of them especially on “ProductRelated_Duration” variable.

2.3. Exploratory Data Analysis (EDA)

2.3.1. Summary of Data

```
summary(data)
```

The output below shows the 5 number summary of each variables:

```
> summary(data)
 administrative   administrative_duration informational
 Min.   : 0.000   Min.   :   0.00         Min.   : 0.0000
 1st Qu.: 0.000   1st Qu.:   0.00         1st Qu.: 0.0000
 Median : 1.000   Median :   7.50         Median : 0.0000
 Mean   : 2.315   Mean   :  80.82         Mean   : 0.5036
 3rd Qu.: 4.000   3rd Qu.:  93.26         3rd Qu.: 0.0000
 Max.   :27.000   Max.   :3398.75         Max.   :24.0000

 informational_duration product_related  product_related_duration
 Min.   :   0.00        Min.   :  0.00   Min.   :    0.0
 1st Qu.:   0.00        1st Qu.:  7.00   1st Qu.:  184.1
 Median :   0.00        Median : 18.00   Median :  598.9
 Mean   :  34.47        Mean   : 31.73   Mean   : 1194.8
 3rd Qu.:   0.00        3rd Qu.: 38.00   3rd Qu.: 1464.2
 Max.   :2549.38        Max.   :705.00   Max.   :63973.5

  bounce_rates        exit_rates       page_values       special_day
 Min.   :0.000000   Min.   :0.00000   Min.   :  0.000   Min.   :0.00000
 1st Qu.:0.000000   1st Qu.:0.01429   1st Qu.:  0.000   1st Qu.:0.00000
 Median :0.003112   Median :0.02516   Median :  0.000   Median :0.00000
 Mean   :0.022191   Mean   :0.04307   Mean   :  5.889   Mean   :0.06143
 3rd Qu.:0.016813   3rd Qu.:0.05000   3rd Qu.:  0.000   3rd Qu.:0.00000
 Max.   :0.200000   Max.   :0.20000   Max.   :361.764   Max.   :1.00000

    month      operating_systems    browser          region
 May    :3364   Min.   :1.000     Min.   : 1.000   Min.   :1.000
 Nov    :2998   1st Qu.:2.000     1st Qu.: 2.000   1st Qu.:1.000
 Mar    :1907   Median :2.000     Median : 2.000   Median :3.000
 Dec    :1727   Mean   :2.124     Mean   : 2.357   Mean   :3.147
 Oct    : 549   3rd Qu.:3.000     3rd Qu.: 2.000   3rd Qu.:4.000
 Sep    : 448   Max.   :8.000     Max.   :13.000   Max.   :9.000
 (Other):1337
  traffic_type             visitor_type    weekend         revenue
 Min.   : 1.00   New_Visitor      : 1694   Mode :logical   Mode :logical
 1st Qu.: 2.00   Other            :   85   FALSE:9462      FALSE:10422
 Median : 2.00   Returning_Visitor:10551   TRUE :2868      TRUE :1908
 Mean   : 4.07
 3rd Qu.: 4.00
 Max.   :20.00
```

Image 2.3.1

Summary of The Data

2.3.2. Distribution of Product Related Duration

Code:

```
ggplot(data, aes(x = product_related_duration)) +
  geom_histogram(fill = "lightblue", color = "black") +
  theme_minimal() +
  labs(title = "Distribution of Product Related Duration",
  x =  \"Product Related Duration", y = "Count")
```

Plot:

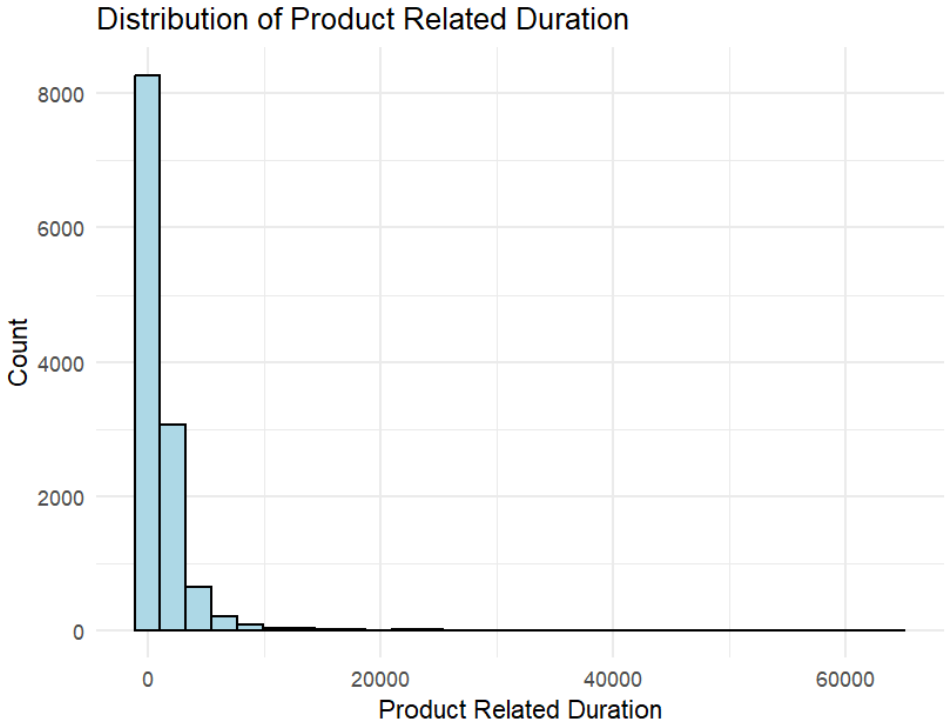


Image 2.3.2

Distribution of Product Related Duration

Interpretation:

This histogram represents the boxplot from the previous graph for outlier. The bars are visually visible and a lot higher on the left side (right-skewed distribution), which shows that most frequently, users spend minimal time on product-related pages during their session. However, there are also many outliers that represent users who spend significantly longer durations, possibly indicating higher engagement or decision-making processes before purchase.

2.3.3. Distribution of Administrative Duration

Code:

```
ggplot(data, aes(x = administrative_duration)) +
  geom_histogram(bins = 30, fill = "lightblue", color = "black",
  alpha = 0.7) + theme_minimal() + labs(title = "Distribution of
```

```
Administrative Duration", x = "Administrative Duration", y =
"Count")
```

Plot:

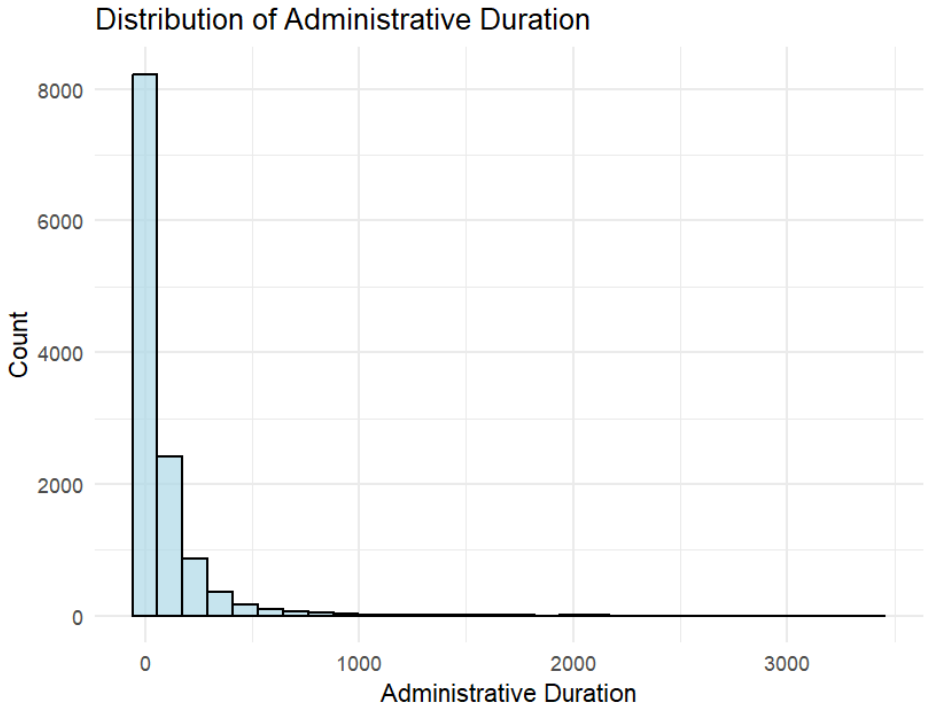


Image 2.3.3

Distribution of Administrative Duration

Interpretation:

The distribution of administrative duration is also right skewed. Users typically spend very little time on administrative pages. The administrative pages likely serve secondary purposes and are not primary areas of interest during sessions. However, we also see that some users have long duration which might indicate usability issues that should be addressed. Moreover, it's best to minimize users' time on the administrative pages to ensure a seamless user experience.

2.3.4. Distribution of Exit Rates

Code:

```
ggplot(data, aes(x = exit_rates)) +
  geom_histogram(bins = 30, fill = "lightblue", color = "black",
  alpha = 0.7) + theme_minimal() +
  labs(title = "Distribution of Exit Rates", x = "Exit Rates", y
  = "Count")
```

Plot:

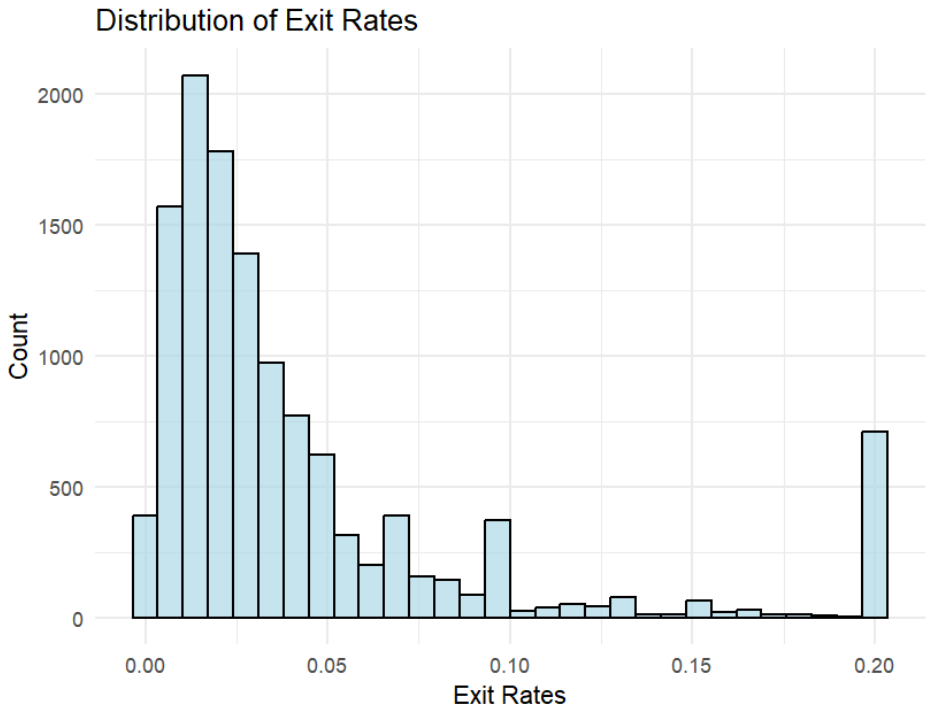


Image 2.3.4

Distribution of Exit Rates

Interpretation:

The distribution of exit rates is nearly fully right skewed but a bimodal with peaks at both near ends. The low exit rates are good because it represents users navigating through multiple pages before exiting, indicating deeper engagement. The high exit rates peak however is bad because it shows that a lot of users also quickly leave the site maybe because the landing page is not too interesting, long loading time, irrelevant displayed content, and more unpredictable reasons that need more research to minimize.

2.3.5. Distribution of Revenue

Code:

```
ggplot(data, aes(x = revenue)) +
  geom_bar(fill = "pink", alpha = 0.75) + theme_minimal() +
  labs(title = "Distribution of Revenue", x = "Revenue", y =
  "Count")
```

Plot:

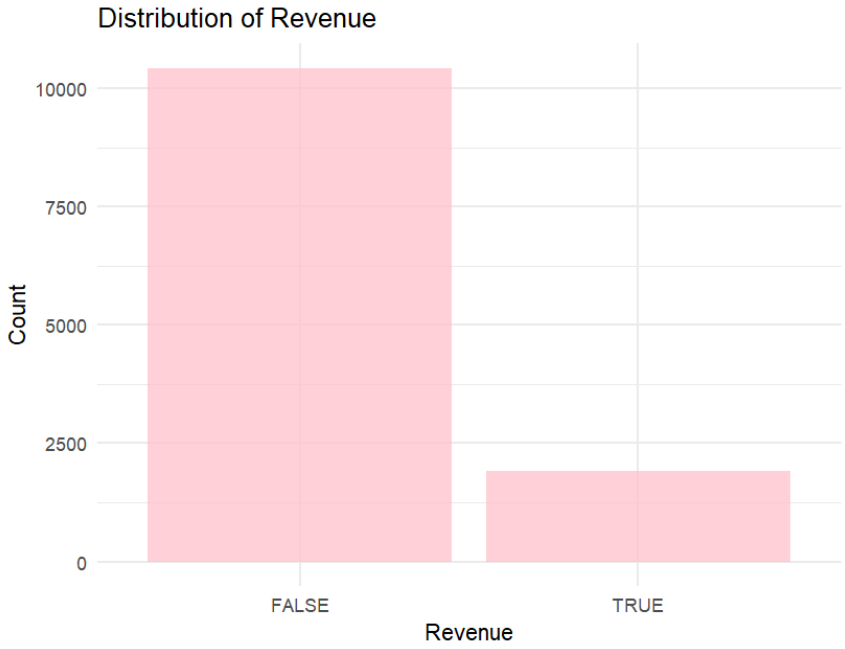


Image 2.3.5

Distribution of Revenue

Interpretation:

There is visible gap between the “FALSE” = no purchase and “TRUE” = yes for a purchase. This shows that most session quite significantly does not result in customer to purchase the product. This imbalance is common in e-commerce, where most sessions don't lead to purchases. But this information is crucial for further analysis to differentiate purchasing users from non-purchasing users.

2.3.6. Count of Visitor Type

Code:

```
ggplot(data, aes(x = visitor_type)) +
  geom_bar(fill = "lightgreen") +
  theme_minimal() +
  labs(title = "Count of Visitor Type", x = "Visitor Type", y =
  "Count")
```

Plot:

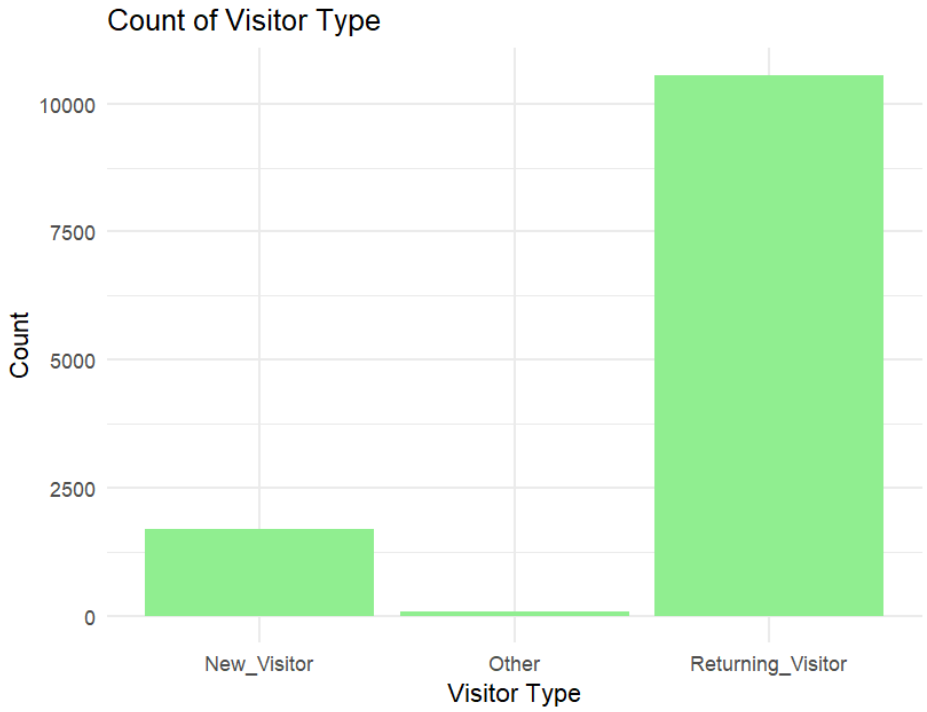


Image 2.3.6

Count of Visitor Type

Interpretation:

The bar chart displays three categories of visitor types. The categories are returning visitors, which dominate the sessions, new visitors, representing a smaller proportion, and a very small proportion of “other” category. Returning visitors indicate strong loyalty and engagement, while new visitors highlight growth potential if effectively converted into repeat users. The "other" category, though minimal, may represent anomalies or unique cases that warrant further investigation. The business needs to focus on retaining and engaging Returning Visitors, improving onboarding for new visitors, and analysing the "other" category to refine tracking or uncover opportunities.

2.3.7. Count of Visits by Month

Code:

```
ggplot(data, aes(x = month)) +
  geom_bar(fill = "lightgreen", color = "black", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Count of Visits by Month", x = "Month", y = "Count")
```

Plot:

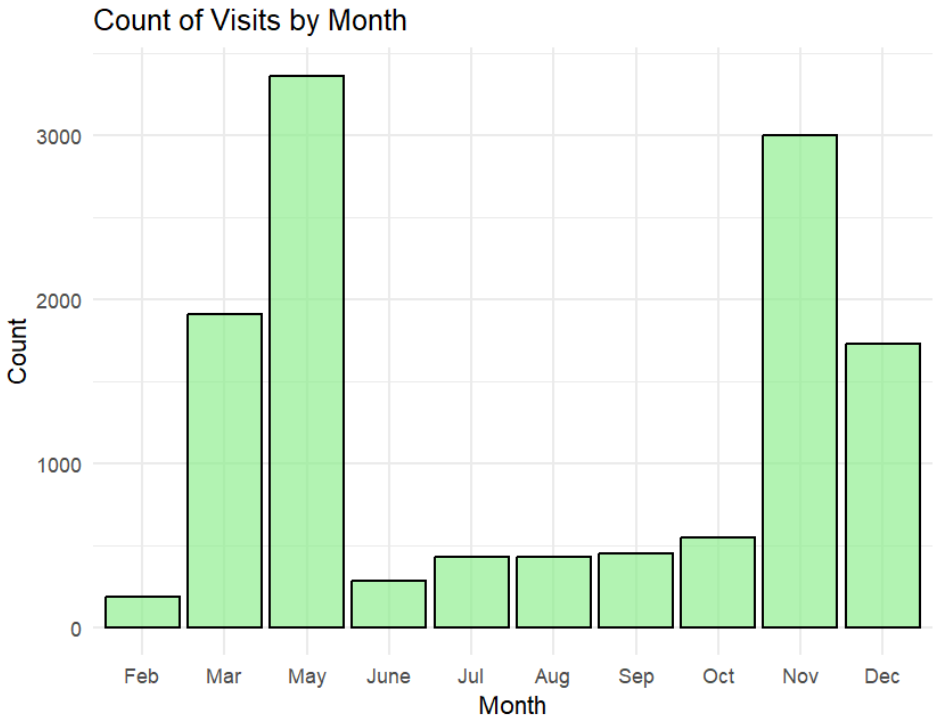


Image 2.3.7

Count of Visits by Month

Interpretation:

The bar chart reveals significant variation in session counts across months, with very high traffic in May and November and quite high traffic in March and December, likely driven by seasonal trends, holidays, or promotional campaigns. The remaining months experience relatively low traffic. The business can strategically target high-traffic months with promotional offers while still exploring strategies to boost engagement during low-traffic periods.

2.3.8. Comparison of Relationship Between Revenue by Visitor Type and by Weekend

Code:

```
?layer_positions
revenue_vs_visitor <- ggplot(data, aes(x = revenue, fill =
  visitor_type)) + geom_bar(position = "stack" , alpha = 0.75) +
  theme_minimal() + labs(title = "Revenue by Visitor Type", x =
  "Revenue", y = "Count")

revenue_vs_weekend <- ggplot(data, aes(x = revenue, fill = weekend))
  + geom_bar(position = "stack" , alpha = 0.75) + theme_minimal()
  + labs(title = "Revenue by Weekend", x = "Revenue", y = "Count")
```

```
library(gridExtra)
grid.arrange(revenue_vs_visitor, revenue_vs_weekend, nrow = 1)
```

Plot:

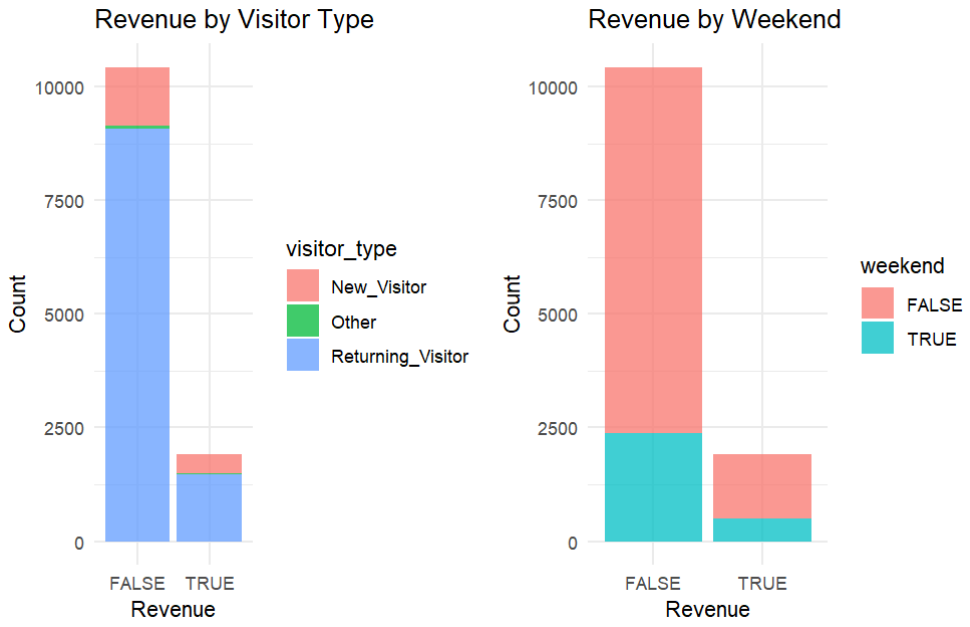


Image 2.3.8

Revenue by Visitor Type and Revenue by Weekend

Interpretation:

Before we interpret this bar, we must keep in mind that the false bar for revenue is higher due to the majority of data not resulting in a purchase. The stacked bar on the left shows that returning visitors contribute a significantly higher proportion of revenue compared to new visitors, indicating that repeat users are more likely to make purchases due to familiarity and trust. On the right graph, on the other hand is quite interesting. Contrary to expectations, revenue is higher during weekdays ("false" for Weekend) than on weekends ("true" for Weekend), despite the perception that users might have more leisure time to shop on weekends. This suggests that purchasing decisions are influenced more by weekday activities, such as work-related shopping or planned purchases.

2.3.9. Relationship Between Revenue and Month

Code:

```
ggplot(data, aes(x = revenue, fill = month)) +
        geom_bar(position = "dodge", alpha = 0.75) +
        theme_minimal() +
```

```
        labs(title = "Revenue by Month", x = "Month", y = "Count")

ggplot(data, aes(x = month, y = revenue, group = 1)) +
        stat_summary(fun = mean, geom = "line", color =
        "darkgreen", size = 1) + theme_minimal() +
        labs(title = "Average Revenue by Month", x = "Month", y =
        "Average Revenue")
```

Plot:

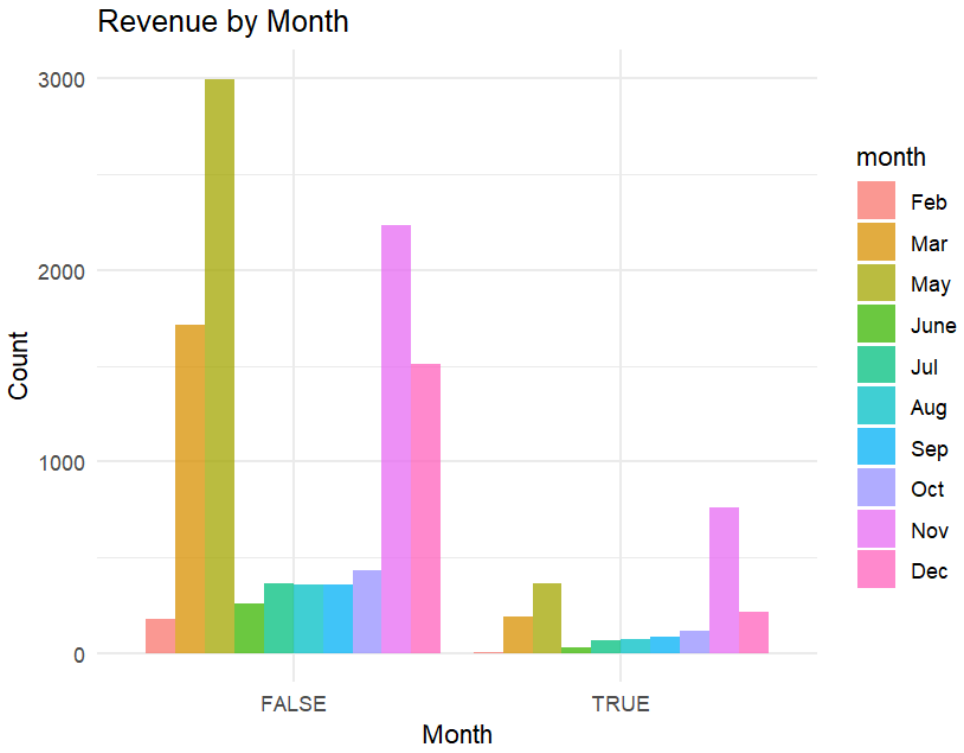


Image 2.3.9

Revenue by Month

This can also be expressed with line graph as follows.

Code:

```
trend_revenue_vs_month <- data.frame(table(data$month,
                          data$revenue))

names(trend_revenue_vs_month) <- c("months", "revenue",
                                   "frequency")

ggplot(trend_revenue_vs_month, aes(x = months, y = frequency)) +
      geom_line(aes(color = revenue, group = revenue), lwd = 1) +
```

```
geom_point(aes(color = revenue, group = revenue, size = 0.1),
show.legend = FALSE) +
scale_color_manual(name = "Revenue", values = c("pink",
"lightgreen")) + theme_light() +
labs(title = "Trend Line for Revenue Based on Months", x =

"Month", y = "Visitors")
```

Plot:

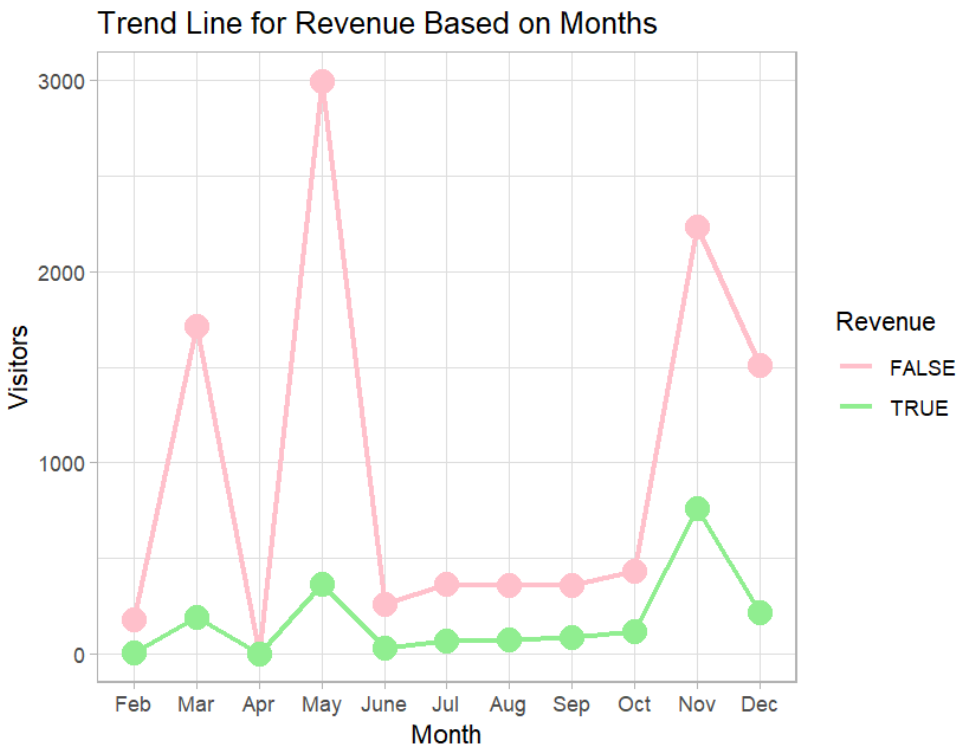


Image 2.3.10

Trend Line for Revenue Based on Months

Interpretation of both graphs:

These graphs are quite similar to the histogram on b. The only difference is the separation of true or false for revenue. It also reveals significant variation in session counts across months, with very high traffic in May and November and quite high traffic in March and December. The remaining months experience relatively low traffic. This applies for both the purchasing and non-purchasing users.

2.3.10. Relationship Between Revenue and and Special Day

Code:

```
ggplot(data, aes(x = special_day, fill = revenue)) +
```

```
  geom_histogram(binwidth = 0.1, position = "dodge", alpha = 0.7)
  + theme_minimal() +
  labs(title = "Revenue by Special Day", x = "Special Day", y =
  "Count")
```

Plot:

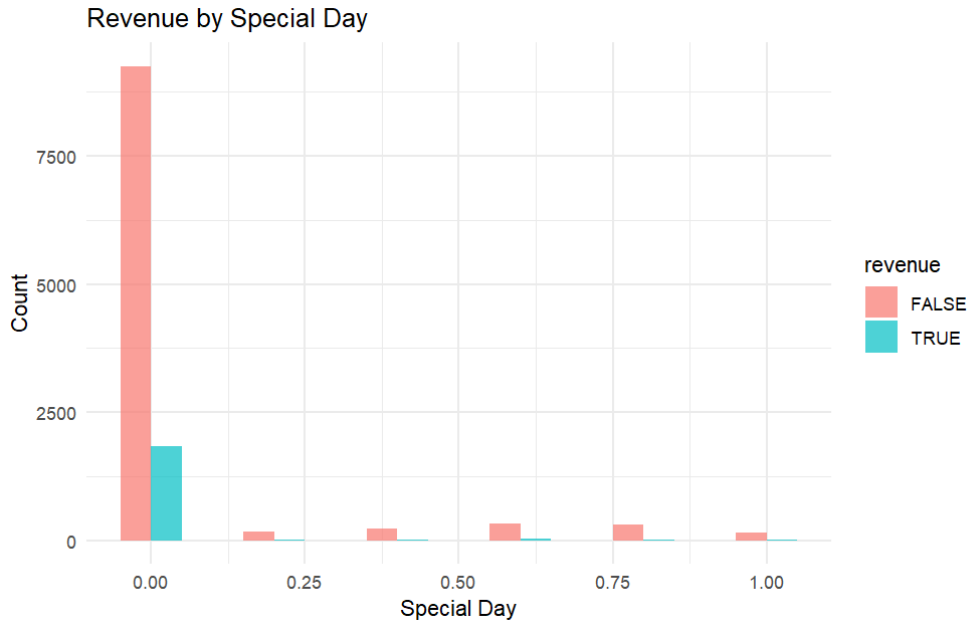


Image 2.3.11

Revenue by Special Day

Interpretation:

For special day in this case, the scale is 0 = not close to 1 = very close. It is again a quite interesting graph because it showed higher revenue on days that are far from special days. This indicates that the closeness to special day does not particularly increase the revenue.

2.3.11. Proportion of Visitor Type by Weekend

Code:

```
visitor_weekend <- data %>%
  count(visitor_type, weekend) %>%
  mutate(Proportion = n / sum(n)) # Adding new column named
  Proportion

ggplot(visitor_weekend, aes(x = visitor_type, y = Proportion, fill
  = weekend)) + geom_bar(stat = "identity") + theme_minimal() +
  labs(title = "Proportion of Visitor Type by Weekend", x =
```

```
  "Visitor Type", y = "Proportion", fill = "Weekend")
```

Plot:

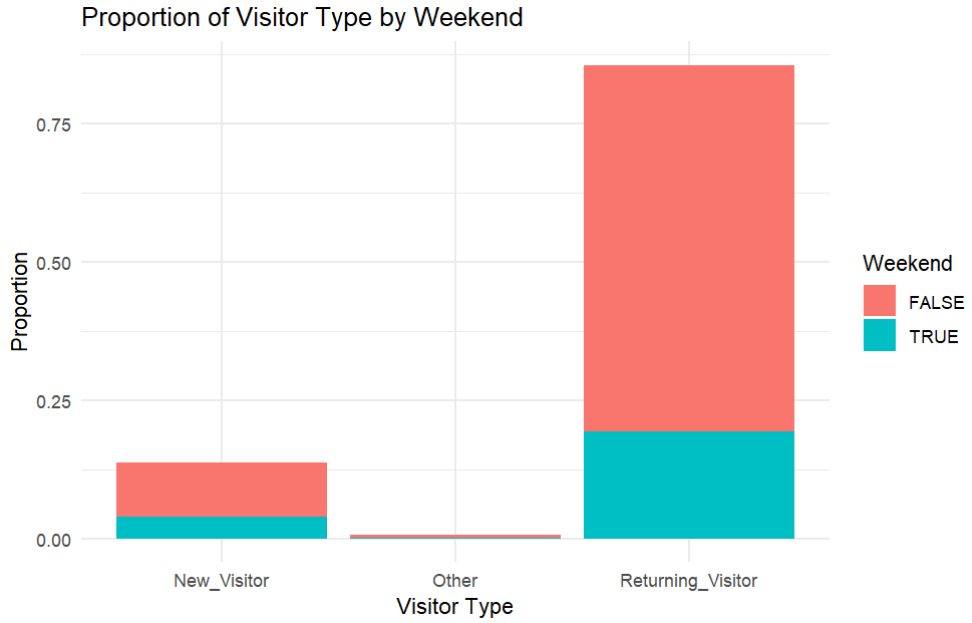


Image 2.3.12

Proportion of Visitor Type by Weekend

Interpretation:

From the graph above, we see that returning visitors actually dominate both weekends and weekdays, though the proportion of new visitors does increase slightly on weekends. So, returning visitors are the primary audience regardless of the day. But the business can try to attract new visitors (especially during weekend when the volume is higher) while continuing to engage with returning visitors.

2.3.12. Correlation between variables

Code:

```
cor_matrix <- cor(numeric_columns)

# Using ggplot
ggplot(cor_melt, aes(Var1, Var2, fill = value)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue", high = "red", mid = "white",
midpoint = 0, limit = c(-1, 1)) +
  theme_minimal() +
  labs(title = "Correlation Heatmap", x = "", y = "") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
# Reshape the cor_matrix into a long-format data frame
cor_melt <- melt(cor_matrix)

# Using corplot
corrplot(cor_matrix, method = "square", type = "lower", diag = TRUE)
```

Plot:

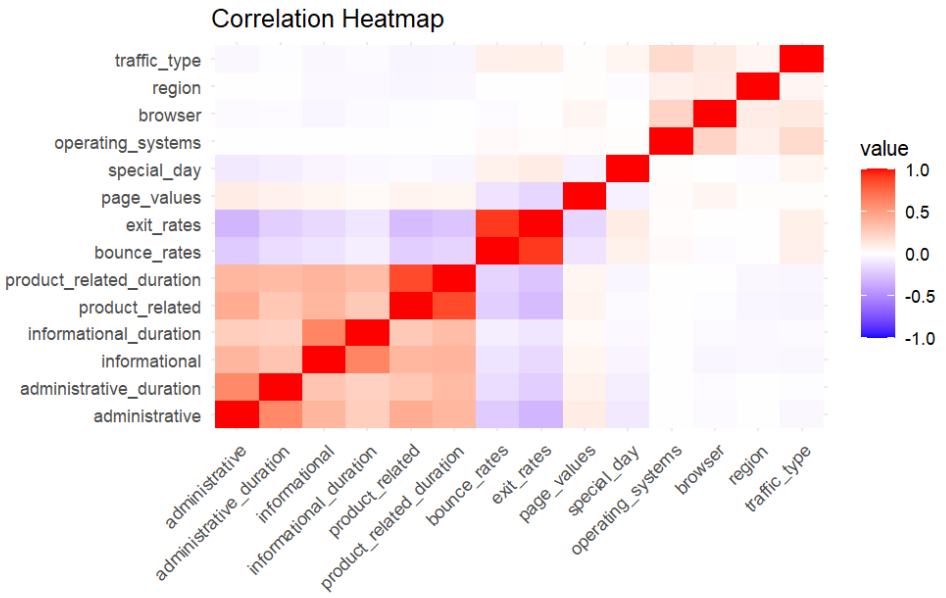


Image 2.3.13

Correlation Heatmap

Interpretation:

The heatmap above shows the correlation between all numerical variables. The strongest correlation is shown with either bright red (positive correlation) or bright purple (negative correlation). So, we observe that exit rates and bounce rates have the higher positive correlation. Product related also have strong positive correlation with product related duration. So is informational with informational duration. This string positive correlation suggests that the higher the value of one variable, the higher the value of the other. Some variables also have negative correlation but not too strong. The strongest negative correlation belongs to the relationship between product related with exit rates.

2.3.13. Relationship between Exit Rates and Bounce Rates by Revenue

Code:

```
ggplot(data, aes(x = bounce_rates, y = exit_rates)) +
```

```
geom_point(aes(color = revenue)) + theme_light() +
labs(title = "Relationship between Exit Rates and Bounce Rates",
x = "Bounce Rates", y = "Exit Rates")
```

Plot:

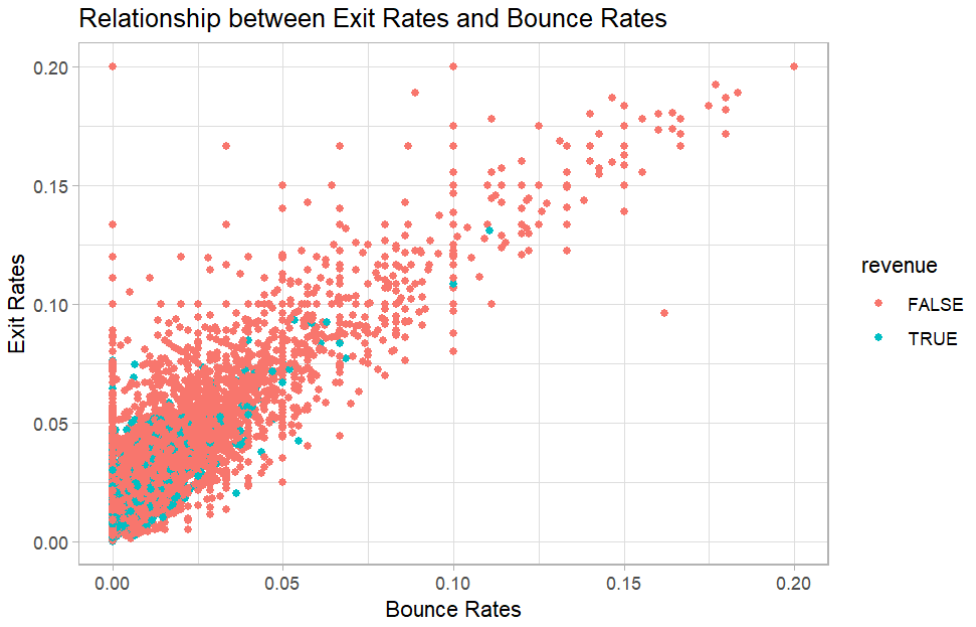


Image 2.3.14

Relationship between Exit Rates and Bounce Rates

Interpretation:

The scatter plot shows a clear positive relationship between exit rates and bounce rates. Sessions without revenue dominate and cluster at higher bounce and exit rates, indicating less engagement, while sessions with revenue appear as a smaller subset, concentrated at lower bounce and exit rates, suggesting deeper user engagement contributes to purchases.

2.3.14. Relationship between Product Related and Product Related Duration

Code:

```
ggplot(data, aes(x = product_related, y =
  product_related_duration))
  + geom_point(aes(color = revenue)) + theme_light() +
  labs(title = "Relationship between Product Related and Product
  Related Duration", x = "Product Related", y = "Product Related
  Duration")
```

Plot:

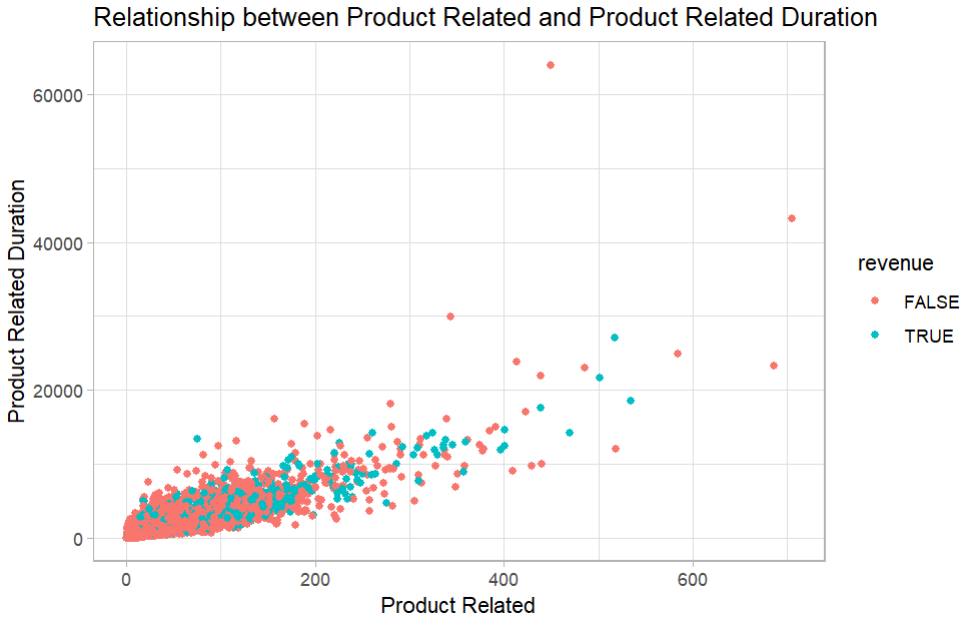


Image 2.3.15

Relationship between Product Related and Product Related Duration

Interpretation:

This scatter plot also shows a positive relationship between product related and product related duration. The graph however does not show a significant difference for sessions with or without revenue in relation to both product related and product related duration. This shows that when user visit the website and search for related product, they spend a long time on that product related page.

2.3.15. Trend Line for Visitor Type Based on Months

Code:

```
trend_visitor_type_vs_month <-
data.frame(table(data$visitor_type, data$month))

names(trend_visitor_type_vs_month) <- c("visitor_type", "month",
"frequency")

ggplot(data = trend_visitor_type_vs_month, mapping = aes(x =
month,
  y = frequency)) + geom_line(mapping = aes(color = visitor_type,
  group = visitor_type), lwd = 1) +
  geom_point(mapping = aes(color = visitor_type, group =
  visitor_type,
```

```
size = 0.1), show.legend = FALSE) + theme_light() +
scale_color_manual(name = "Visitor Type", values =
c("lightblue", "pink", "lightgreen")) +
labs(title = "Trend Line for Visitor Type Based on Months", x =
"Months", y = "Visitors")
```

Plot:

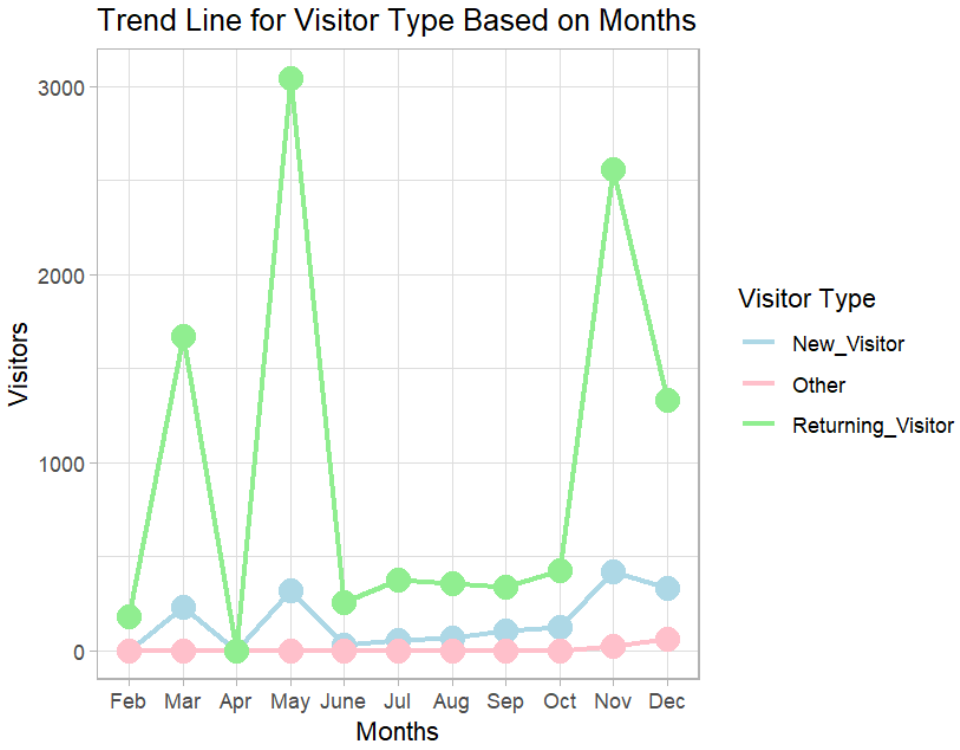


Image 2.3.16

Trend Line for Visitor Type Based on Months

Interpretation:

The graph above shows that there is not too much difference between the trend of new visitor and returning visitor throughout the years. Although new visitor has the highest peak on December and returning visitor in May. In general, both have three visible peaks in March, May, and November when the business is at high traffic. For the "other" category, however, the line is pretty straight, very close to zero too. This shows that it's just come rare occurrences that sometimes pop at whenever time of the year.

2.4. Preparation Before Building Models

2.4.1. Checking type of variables

Code:

```
str(data)
```

Output:

```
'data.frame': 12330 obs. of  18 variables:
 $ administrative: num  -0.697 -0.697 -0.697 -0.697 -0.697 ...
 $ administrative_duration: num  -0.457 -0.457 -0.457 -0.457 -
   0.457 ...
 $ informational: num  -0.396 -0.396 -0.396 -0.396 -0.396 ...
 $ informational_duration: num  -0.245 -0.245 -0.245 -0.245 -
   0.245 ...
 $ product_related: num  -0.691 -0.668 -0.691 -0.668 -0.489 ...
 $ product_related_duration: num  -0.624 -0.591 -0.624 -0.623 -
   0.296 ...
 $ bounce_rates: num  3.667 -0.4577 3.667 0.5735 -0.0452 ...
 $ exit_rates: num  3.229 1.171 3.229 1.995 0.143 ...
 $ page_values: num  -0.317 -0.317 -0.317 -0.317 -0.317 ...
 $ special_day: num  -0.309 -0.309 -0.309 -0.309 -0.309 ...
 $ month: Factor w/ 11 levels "Feb","Mar","Apr",..: 1 1 1 1 1 1
   1 1 1 1 ...
 $ operating_systems: int  1 2 4 3 3 2 2 1 2 2 ...
 $ browser: int  1 2 1 2 3 2 4 2 2 4 ...
 $ region: int  1 1 9 2 1 1 3 1 2 1 ...
 $ traffic_type: int  1 2 3 4 4 3 3 5 3 2 ...
 $ visitor_type: Factor w/ 3 levels "New_Visitor",..: 3 3 3 3 3
   3 3 3 3 3 ...
 $ weekend: logi  FALSE FALSE FALSE FALSE TRUE FALSE ...
 $ revenue: Factor w/ 2 levels "FALSE","TRUE": 1 1 1 1 1 1 1 . . .
```

2.4.2. Checking completeness of data

```
sum(is.na(data))
[1] 0
```

2.4.3. Converting non-numerical variables to factors

```
data$month <- as.factor(data$month)
data$visitor_type <- as.factor(data$visitor_type)
data$revenue <- as.factor(data$revenue)
```

2.4.4. Normalizing numerical columns for standardization

```
num_cols <- c("administrative", "administrative_duration",
              "informational", "informational_duration",
              "product_related", " product_related_duration",
              "bounce_rates", "exit_rates", "page_values",
              "special_day")
data[num_cols] <- scale(data[num_cols])
```

2.4.5. Splitting data to training and testing data (70% & 30% respectively) using seed = 123 (for reproducibility).

```
library(caret)
set.seed(123)
trainIdx <- createDataPartition(data$revenue, p = 0.7, list = FALSE)
train_data <- data[trainIdx,]
test_data <- data[-trainIdx,]
```

2.5. Building Supervised Learning Models

There are many types of supervised learning models. Selecting the appropriate classification algorithm is essential because each one has its own advantages and limitations, and no single algorithm is optimal for all types of problems. The decision is influenced by factors such as the size, quality, and characteristics of the data, the time sensitivity of the task, and the intended use of the results.

Accuracy of the model will be counted with confusion matrix, which is a tool that helps assess the performance of a classification model. It contains the following components:

- True Positives (TP): The number of instances where the model correctly predicted the positive class.
- True Negatives (TN): The number of instances where the model correctly predicted the negative class.
- False Positives (FP): The number of instances where the model incorrectly predicted the positive class (i.e., it predicted positive when the actual class was negative).
- False Negatives (FN): The number of instances where the model incorrectly predicted the negative class (i.e., it predicted negative when the actual class was positive).

Then, the accuracy is calculated with the formula:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

The models that we will be making are as follows.

2.5.1. Decision Tree

A Decision Tree is a common method for classifying data. It starts with one main decision point and then branches out into different options or outcomes based on the data. It's like a flowchart where each branch represents a possible decision or action, and each outcome leads to another decision until the final result is reached. This method is useful because it clearly shows how decisions are made step by step.

Code:

```
library(rpart)
tree_model <- rpart(revenue ~ ., data = train_data, method =
              'class')
library(rpart.plot)
rpart.plot(tree_model, main = 'Decision Tree for r')
prediction_tree <- predict(tree_model, test_data, type = 'class')
cm_tree <- confusionMatrix(prediction_tree, test_data$revenue);
cm_tree
```

Decision Tree visualization:

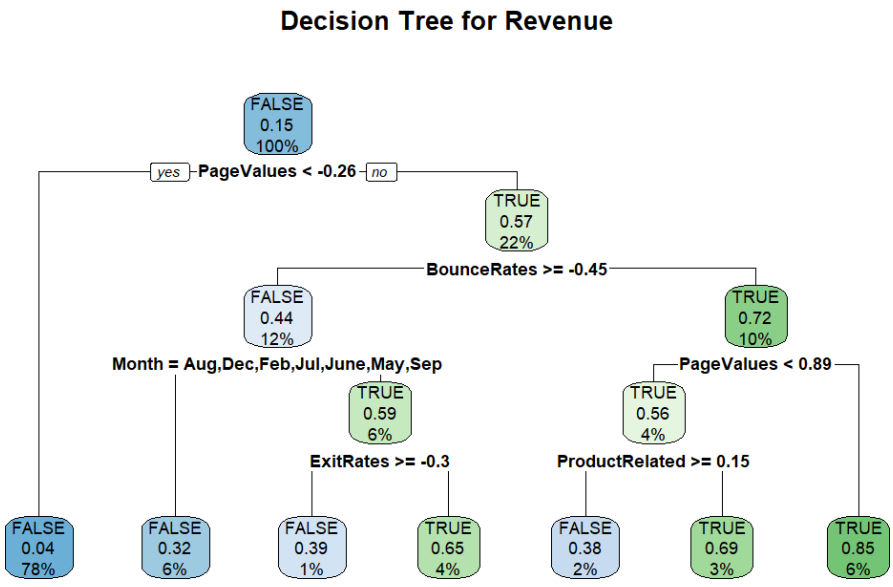


Image 2.5.1

Decision Tree for Revenue

Confusion Matrix Output:

```
Confusion Matrix and Statistics

          Reference
Prediction FALSE TRUE
     FALSE  2996  244
     TRUE    130  328

               Accuracy : 0.8989
                 95% CI : (0.8887, 0.9084)
    No Information Rate : 0.8453
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.579

 Mcnemar's Test P-Value : 5.124e-09

            Sensitivity : 0.9584
            Specificity : 0.5734
         Pos Pred Value : 0.9247
         Neg Pred Value : 0.7162
             Prevalence : 0.8453
         Detection Rate : 0.8102
   Detection Prevalence : 0.8761
      Balanced Accuracy : 0.7659

       'Positive' Class : FALSE
```

Image 2.5.2

Confusion Matrix and Statistics

Interpretation:

We see from the confusion matrix and statistics above that 3324 data are classified correctly out of 3698 data. The accuracy is 89.89% with confidence interval of 95%. So, the true accuracy of this model lies between 87.21% and 89.31%. The accuracy we would expect if we always predicted the most frequent class (No Information Rate -> NIR) is 84.53%. As the model accuracy is better than the NIR, the model is providing useful predictions. This is tested with p value. The very small p-value (close to 0) indicates that the model's accuracy is statistically significantly better than just predicting the most frequent class, which is a good sign.

Kappa statistic measures the agreement between the predicted and actual values that ranges from -1 (perfect disagreement) to 1 (perfect agreement). The value 0.4535 means moderate agreement. This model's predictions are somewhat reliable, but there is room for improvement. McNemar's test checks whether the misclassifications are statistically significantly different. The value <2.2e-16 is so small and proves that the misclassification is not a lot. Sensitivity or recall measures the model's ability to correctly identify positive cases (True Positives). A sensitivity of 97.12% means the model is very good at detecting the positive class. Specificity measures the ability of the model to

correctly identify negative cases (True Negatives). A specificity of 40.03% indicates that the model struggles to correctly classify negative cases and tends to make errors here.

Positive Predictive Value (Precision) is the proportion of predicted positives that were actually positive. In this case, 89.85% is fairly good. But the Negative Predictive Value is 71.79%, which is moderate. Prevalence represents the proportion of actual positive instances in the dataset. Here, 84.53% of the data belongs to the positive class. The detection rate is the proportion of actual positives that were correctly detected by the model. Detection prevalence is the proportion of positive predictions made by the model. Balanced Accuracy is the average of sensitivity and specificity. It helps to give a better measure of accuracy when the class distribution is imbalanced. 68.58% suggests that the model has moderate performance across both classes.

What we will look out to from this decision tree model is as follows.

- 3324 data are classified correctly out of 3698 data.
- True accuracy = between 87.21% and 89.31%.
- p-value = close to 0 -> indicates that the model's accuracy is statistically useful.
- Sensitivity or recall = 97.12% -> very good at detecting the positive class.
- Specificity = 40.03% -> struggles to correctly classify negative cases and tends to make errors here.

2.5.2. Random Forest

Random Forest is a method that uses many decisions to make better predictions. It creates a “forest” of trees, each one making its own prediction. Then, the final decision is based on what most of the trees say. This helps improve accuracy because it avoids mistakes made by any one tree alone, making it more reliable with complex data.

Code:

```
library(randomForest)
rf_model <- randomForest(revenue ~ . , data = train_data, ntree =
            100, importance = TRUE)
# Making predictions on test data
prediction_rf <- predict(rf_model, test_data)
# Creating a confusion matrix to see accuracy of model
cm_rf <- confusionMatrix(prediction_rf,test_data$revenue); cm_rf
# Model is used to predict whether a customer ended a session
with a transaction (making a purchase). It uses input features to
```

```
identify patterns and predict the likelihood of a transaction.
important <- importance(rf_model); important
varImpPlot(rf_model)
```

Random Forest Visualization:

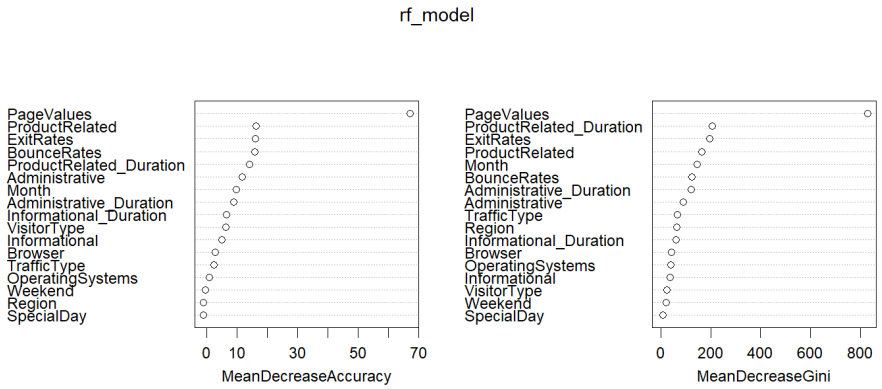


Image 2.5.3

Random Forest Visualization

Confusion Matrix Output:

```
Confusion Matrix and Statistics

          Reference
Prediction FALSE TRUE
     FALSE  3010  240
     TRUE    116  332

               Accuracy : 0.9037
                 95% CI : (0.8938, 0.9131)
    No Information Rate : 0.8453
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.5961

 Mcnemar's Test P-Value : 7.078e-11

            Sensitivity : 0.9629
            Specificity : 0.5804
         Pos Pred Value : 0.9262
         Neg Pred Value : 0.7411
             Prevalence : 0.8453
         Detection Rate : 0.8140
   Detection Prevalence : 0.8789
      Balanced Accuracy : 0.7717

       'Positive' Class : FALSE
```

Image 2.5.4

Confusion Matrix Output of Random Forest

Interpretation:

- 3342 data are classified correctly out of 3698 data.
- True accuracy = between 89.38% and 91.31%.
- p-value = close to 0 -> indicates that the model's accuracy is statistically useful.

- Sensitivity or recall = 96.29% -> very good at detecting the positive class.
- Specificity = 58.04% -> moderate at classifying negative cases.

2.5.3. SVM (Support Vector Machine)

SVM (Support Vector Machine) is a powerful method that helps separate different categories of data by finding a boundary that best divides them. It's good for both simple and complex problems, even when the data isn't easily separable. SVM works well in tasks like image recognition or text classification, but it can be slow when dealing with large amounts of data.

Code:

```
library(e1071)
classifier <- svm(revenue ~., data = train_data, type ='C-
classification', kernel = 'linear')
prediction_svm <- predict(classifier, test_data)
cm_svm <- confusionMatrix(prediction_svm, test_data$revenue);
```

Confusion Matrix Output:

```
Confusion Matrix and Statistics

          Reference
Prediction FALSE TRUE
     FALSE  3058  356
     TRUE     68  216

               Accuracy : 0.8853
                 95% CI : (0.8746, 0.8954)
    No Information Rate : 0.8453
    P-Value [Acc > NIR] : 1.586e-12

                  Kappa : 0.448

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9782
            Specificity : 0.3776
         Pos Pred Value : 0.8957
         Neg Pred Value : 0.7606
             Prevalence : 0.8453
         Detection Rate : 0.8269
   Detection Prevalence : 0.9232
      Balanced Accuracy : 0.6779

       'Positive' Class : FALSE
```

Image 2.5.5

Confusion Matrix Output of Support Vector Machine

Interpretation:

- 3274 data are classified correctly out of 3698 data.

- True accuracy = between 87.46% and 89.54%.
- p-value = close to 0 -> indicates that the model's accuracy is statistically useful.
- Sensitivity or recall = 97.82% -> very good at detecting the positive class.
- Specificity = 37.76% -> struggles to correctly classify negative cases and tends to make errors here.

2.5.4. Naïve-Bayes Classifier

Naïve Bayes Classifier is a fast and efficient classification method that predicts the category of something based on the likelihood of its features. It assumes that the features are independent of each other, which makes it simple and quick, especially for large datasets. It works well for tasks like spam email filtering or categorizing news articles, but its performance can drop if the features are related to each other.

Code:

```
library(caTools)
nb_model <- naiveBayes(revenue ~ ., data = train_data)
prediction_nb <- predict(nb_model, test_data)
cm_nb <- confusionMatrix(test_data$revenue,prediction_nb); cm_nb
```

Confusion Matrix Output:

```
Confusion Matrix and Statistics

          Reference
Prediction FALSE TRUE
     FALSE  2589  537
     TRUE    202  370

               Accuracy : 0.8002
                 95% CI : (0.7869, 0.8129)
    No Information Rate : 0.7547
    P-Value [Acc > NIR] : 2.96e-11

                  Kappa : 0.3834

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9276
            Specificity : 0.4079
         Pos Pred Value : 0.8282
         Neg Pred Value : 0.6469
             Prevalence : 0.7547
         Detection Rate : 0.7001
   Detection Prevalence : 0.8453
      Balanced Accuracy : 0.6678

       'Positive' Class : FALSE
```

Image 2.5.6

Confusion Matrix Output of Naïve-Bayes Classifier

Interpretation:

- 2959 data are classified correctly out of 3698 data.
- True accuracy = between 78.69% and 81.29%.
- p-value = close to 0 -> indicates that the model's accuracy is statistically useful.
- Sensitivity or recall = 92.76% -> very good at detecting the positive class.
- Specificity = 40.79% -> struggles to correctly classify negative cases and tends to make errors here.

2.5.5. KNN Algorithm

K-Nearest Neighbor (KNN) is a simple algorithm that classifies data based on the closest data points. When given new data, it checks which points in the training data are nearest, and then it picks the most common class among those neighbors. It's easy to use, but it can take a long time with large datasets since it needs to compare the new data with all the others.

Code:

```
library(class)
# Normalize the data
normalize <- function(x) {
  return((x - min(x)) / (max(x) - min(x)))
}
data_normalized <- as.data.frame(lapply(data[, num_cols],
normalize))
data_normalized$revenue <- data$revenue
data_normalized$revenue <- as.factor(data_normalized$revenue)
train_indices <- createDataPartition(data_normalized$revenue, p =
0.7, list = FALSE)
train_data2 <- data_normalized[train_indices, ]
test_data2 <- data_normalized[-train_indices, ]
train_features <- train_data2[, -ncol(train_data2)]  # All
columns except Revenue
train_labels <- train_data2$revenue
test_features <- test_data2[, -ncol(test_data2)]
test_labels <- test_data2$revenue
# Prediction & Confusion matrix
k <- 15
```

```
predictions <- knn(train_features, test_features, train_labels, k
               = k)
cm_knn <- confusionMatrix(predictions, test_labels); cm_knn
# Finding optimal k value
accuracy_values <- c()
for (k in 1:20) {
  predictions <- knn(train_features, test_features, train_labels,
  k = k)
  cm <- confusionMatrix(predictions, test_labels)
  accuracy_values <- c(accuracy_values, cm$overall["Accuracy"])
}

# Plot accuracy vs. k
plot(1:20, accuracy_values, type = "b", xlab = "k", ylab =
"Accuracy", main = "Optimal k Selection"
```

Confusion Matrix Output:

```
Confusion Matrix and Statistics

          Reference
Prediction FALSE TRUE
     FALSE  3036  343
     TRUE     90  229

               Accuracy : 0.8829
                 95% CI : (0.8721, 0.8931)
    No Information Rate : 0.8453
    P-Value [Acc > NIR] : 3.356e-11

                  Kappa : 0.4535

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9712
            Specificity : 0.4003
         Pos Pred Value : 0.8985
         Neg Pred Value : 0.7179
             Prevalence : 0.8453
         Detection Rate : 0.8210
   Detection Prevalence : 0.9137
      Balanced Accuracy : 0.6858

       'Positive' Class : FALSE
```

Image 2.5.7

Confusion Matrix Output of KNN Algorithm

Interpretation:

- 3265 data are classified correctly out of 3698 data.

- True accuracy = between 87.21% and 89.31%.
- p-value = close to 0 -> indicates that the model's accuracy is statistically useful.
- Sensitivity or recall = 97.12% -> very good at detecting the positive class.
- Specificity = 40.03% -> struggles to correctly classify negative cases and tends to make errors here.

Plot accuracy in relation to optimal value k:

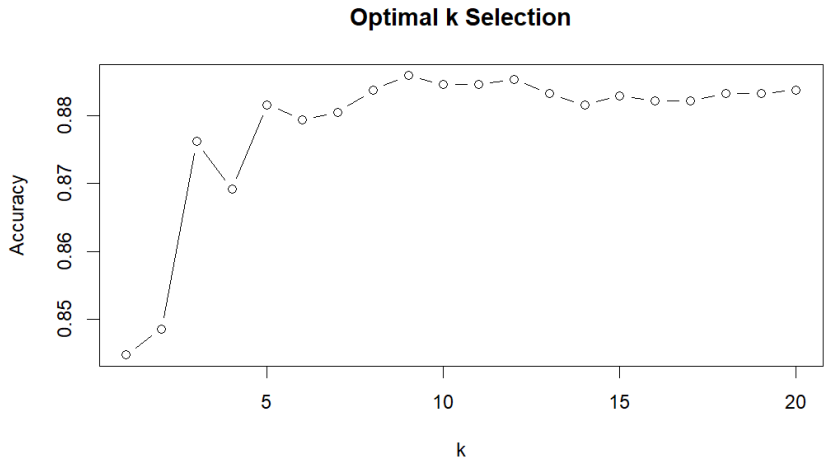


Image 2.5.8
Optimal k Selection

2.5.6. Logistic Regression

Logistic Regression is another classification algorithm, but it belongs to a different category compared to Decision Trees, Random Forests, KNN, etc that are mentioned above, which are based on different approaches for classification and clustering, Logistic Regression is a statistical method used for binary classification to predict the probability of a binary outcome. Instead of making a direct prediction like most models, Logistic Regression estimates the probability that a given data point belongs to a particular class. It works by fitting a model to the data that maps input features to a probability value, which is then used to classify the data point. This makes it ideal for situations where the relationship between the input variables and the target variable is not linear.

Code:

```
log_model <- glm(revenue ~., data = train_data, family =
binomial)
```

```
prediction_log <- predict(log_model, test_data, type =
'response')
class_prediction <- ifelse(prediction_log > 0.5, 'TRUE', 'FALSE')

cm_log <- confusionMatrix(as.factor(class_prediction),
          test_data$revenue); cm_log
```

Confusion Matrix Output:

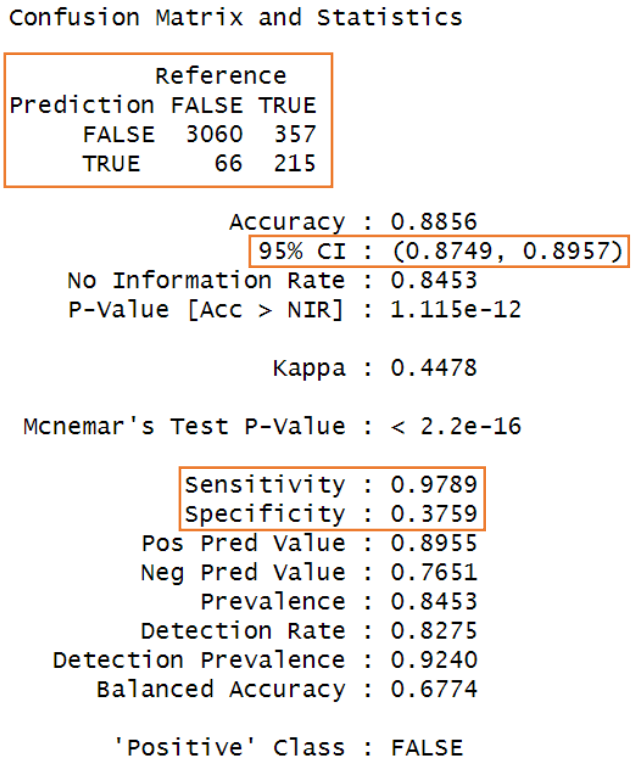


Image 2.5.9

Confusion Matrix Output of Logistic Regression

Interpretation:

- 3275 data are classified correctly out of 3698 data.
- True accuracy = between 87.49% and 89.57%.
- p-value = close to 0 -> indicates that the model's accuracy is statistically useful.
- Sensitivity or recall = 97.89% -> very good at detecting the positive class.
- Specificity = 37.59% -> struggles to correctly classify negative cases and tends to make errors here.

2.5.7. Comparing Supervised Learning Models

Now, we are comparing the accuracy of models that have been made before. This helps us decide which model is the most suitable for our dataset.

Code:

```
acc_tree <- cm_tree$overall[1]*100; acc_tree
acc_rf <- cm_rf$overall[1]*100; acc_rf
acc_svm <- cm_svm$overall[1]*100; acc_svm
acc_nb <- cm_nb$overall[1]*100; acc_nb
acc_knn <- cm_knn$overall[1]*100; acc_knn
acc_log <- cm_log$overall[1]*100; acc_log

column_name <- c('Decision.Tree', 'Random.Forest', 'SVM',
"Naive.Bayes", 'KNN', 'Logistic.Regression')
acc_vector <- c(acc_tree, acc_rf, acc_svm, acc_nb, acc_knn,
acc_log)
acc_vector <- round(acc_vector, digits = 2); acc_vector
acc_data <- data.frame(t(acc_vector)); acc_data
colnames(acc_data) <- column_name
rownames(acc_data) <- 'Accuracy'
library(knitr)
kable(acc_data, align = 'c')
```

Output:

	Decision.Tree	Random.Forest	SVM	Naive.Bayes	KNN	Logistic.Regression
Accuracy	89.89	90.37	88.53	80.02	88.29	88.56

Image 2.5.10

Comparison of Supervised Learning Models

Interpretation:

From the comparison table above, we observe that random forest is the best supervised learning models with the high accuracy of 90.37%. The close second is decision tree (simpler version of random forest) with similar accuracy of 89.89%. Some other models are also quite accurate such as Logistic Regression at 66.56%, SVM at 88.53%, and KNN at 88.29%. The Naïve Bayes Classifier, however, have somewhat lower accuracy at 88.02%. This indicates that the model is not suitable for this type of analysis.

Chapter 3. Closing

3.1. Summary

The dataset, titled Online Shoppers Purchasing Intention Dataset, contains 12,331 rows representing individual sessions over a one-year period. The dataset aims to avoid biases by ensuring each session belongs to a unique user, regardless of specific campaigns or user profiles. Notably, 84.5% of the sessions (10,422) did not result in a purchase, while only 15.5% (1,908) ended in shopping. The dataset includes various features such as page visit counts, durations, bounce rates, exit rates, and categorical variables like the Month, Visitor Type, and Weekend Activity, alongside a binary target variable indicating purchase outcomes.

The dataset was loaded into R and checked for missing values, revealing none. Column names were cleaned using the clean names() function to ensure consistency. Categorical variables, such as “Month” and “VisitorType,” were converted to factors for better analysis and visualization. Duplicate checking was deemed unnecessary since each row represents a unique session. Outliers were identified using the IQR method, revealing notable outliers in variables like “ProductRelated_Duration” and “PageValues,” which were visualized with boxplots.

The EDA (Exploratory Data Analysis) provided insights into variable distributions. Variables like "ProductRelated_Duration" and "Administrative_Duration" exhibited right-skewed distributions, indicating that most users spent minimal time on related pages, with outliers reflecting users with higher engagement. Exit rates showed a bimodal distribution, suggesting variability in user behaviour, with both deep engagement and quick exits. Revenue distribution highlighted a significant imbalance, with far fewer sessions resulting in purchases, which aligns with typical e-commerce trends.

Returning visitors dominated the sessions, contributing significantly to revenue compared to new visitors, which demonstrates the importance of user loyalty. Interestingly, revenue was higher on weekdays than weekends, contradicting the common assumption that users shop more during their leisure time. This suggests that weekday purchases might be influenced by work-related needs or planned activities. Monthly traffic varied significantly, peaking in May and November, likely driven by seasonal trends and promotions.

The correlation heatmap revealed strong positive correlations, such as between "ProductRelated" and "ProductRelated_Duration," and "BounceRates" with "ExitRates." Negative correlations, though weaker, were observed, with the strongest being between "ProductRelated" and "ExitRates," indicating that higher engagement on product pages correlates with lower exit rates. A scatter plot showed that sessions with lower bounce and exit rates were more likely to result in revenue, reflecting the importance of user engagement for conversions.

Interestingly, revenue was higher on days far from special occasions, indicating that proximity to special days does not necessarily boost sales. This insight challenges typical marketing strategies focused on special-day campaigns. Returning visitors remained the primary

audience across both weekdays and weekends, with a slight increase in new visitor proportion during weekends, highlighting an opportunity to attract and convert new users.

3.2. Conclusion

Analysing the Online Shoppers Purchasing Intention Dataset provides valuable insights into user behaviour and purchasing patterns in an e-commerce context. Key findings highlight the importance of focusing on returning visitors, weekday activity, and user engagement metrics to drive revenue. Business suggestions are explained as follows.

- Returning visitors are the primary revenue contributors, demonstrating the value of building and maintaining customer loyalty. The business should invest in retention strategies, such as personalized offers, loyalty programs, and targeted re-engagement campaigns.
- The unexpected trend of higher revenue on weekdays suggests an opportunity to optimize weekday marketing efforts. The business could implement workday promotions or advertise productivity related products during these periods to align with user intent.
- Low bounce and exit rates were closely linked to increased purchases, emphasizing the need to enhance the user experience. The business should focus on reducing friction in the customer journey by improving website navigation, ensuring fast load times, and creating engaging product content to keep users on-site.
- Revenue does not peak near special days challenges traditional marketing assumptions. Instead of over reliance on special day campaigns, the business could explore strategies that sustain user engagement and conversions throughout the year, leveraging data-driven promotions based on user behaviour.

By implementing these actionable recommendations, e-commerce businesses can better align their strategies with observed customer behaviours, improving overall performance and driving sustained growth.